ECOLE POLYTECHNIQUE DE BRUXELLES

2025-2026

**INFO-H503 - GPU computing**

# Mini project : study of a simple GPU kernel


Mandel Eliot 493519

Bonatto Daniele

Yildirim Emirhan 459800

Soetens Elise


17 mars 2026

# Table des matières

**1 Introduction** **1**

**2 Computer and GPU specificities** **2**

**3 Analysis** **3**

3.1 Memory studies . . . . . 3

3.2 Kernel execution on multiple elements . . . . . 4

3.3 Roofline analysis . . . . . 6

**4 Conclusion** **7**

# 1 Introduction

As part of this project, we were asked to analyse and compare two basic operations that take place within a GPU core. We therefore chose **float addition** and **float multiplication**. To do this, we began by conducting a performance analysis, varying the size of the array and, consequently, the influence of memory on the computation. Next, we studied the application of the core to multiple elements and, finally, the influence of intensity on computational power. We will see later that the results are largely similar between addition and multiplication for this type of data.

# 2 Computer and GPU specificities

We used a personal computer for this project; the GPU and its specifications are therefore as follows :

- GPU NAME,NVIDIA GeForce GTX : 1050
- GPU CC : 6.1
- GPU SM COUNT : 5
- GPU MAX THREADS PER BLOCK : 1024
- GPU WARP SIZE : 32
- GPU GLOBAL MEM GB :1.99988
- GPU SHARED MEM PER BLOCK KB : 48
- GPU REGS PER BLOCK : 65536
- THREADS PER BLOCK = 256 (We fixed 256 threads per block, a standard CUDA choice and a multiple of the warp size 32 as asked on the course)

# 3 Analysis

In this section, we will look at the key differences between addition and multiplication calculations on floats datas types and how the GPU perform these operations.We will also analyse how these factors affect everyday computer use. The GPU runtime is measured with cudaEvent around the kernel launch. Therefore, the reported GPU time corresponds to the kernel execution time only

## 3.1 Memory studies

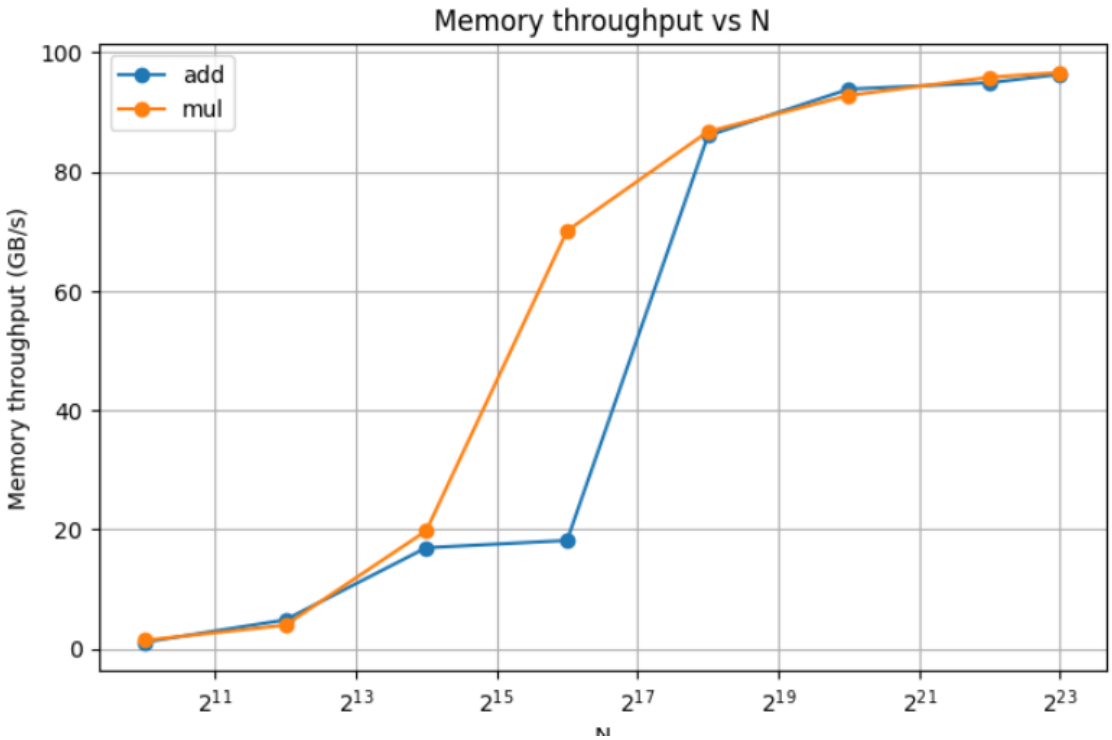


FIGURE 3.1: Graph of the memory througput depending of the size of the array N

We can see here that our curve levels off on two occasions. For small values of N, performance is poor due to the fixed cost of launching the kernel and the under-utilisation of the GPU. Howe-

ver, when the array size N reaches a very large value (approximately $2^{20}$), the kernel reaches the maximum bandwidth of the global memory. This is entirely logical given the limitations of this relatively simple type of kernel and this kind of operation. The only difference in this graph is likely due to measurement noise or minor execution differences, since both kernels have the same memory access pattern and operate under the same conditions.

## 3.2 Kernel execution on multiple elements

We use a neighboring-elements mapping like we saw on the second TP : each thread computes its global index `tid`, starts at `start = tid * j`, and processes the consecutive elements `start`, `start+1, ..., start+(j-1)`.

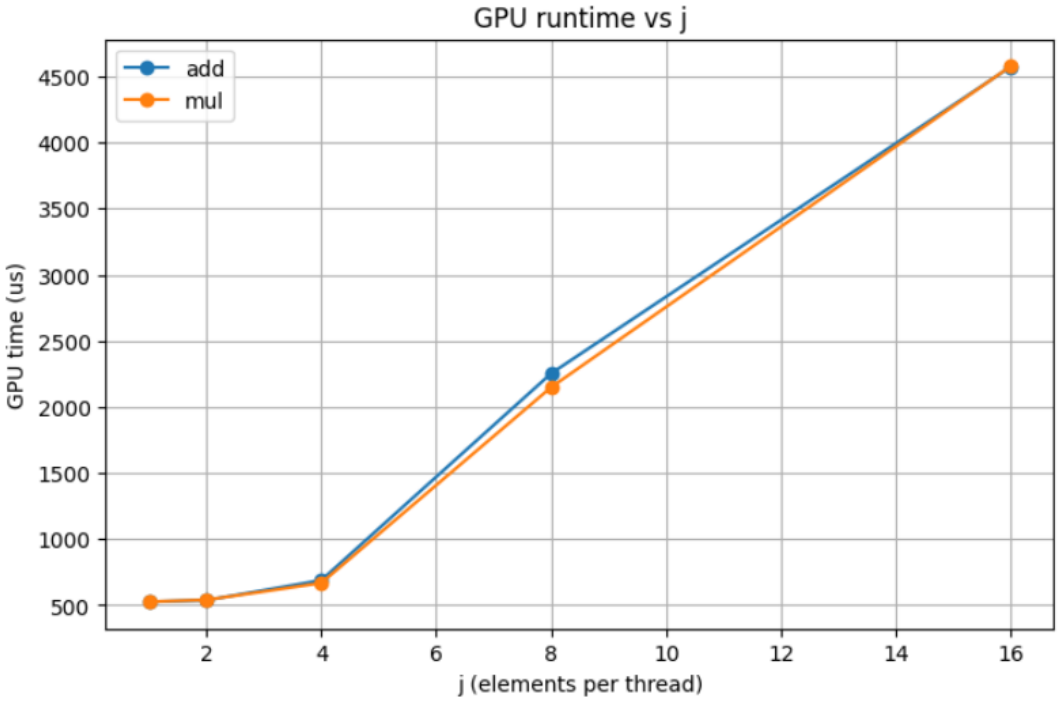


FIGURE 3.2: Execution time of the GPU depending of the number of element on which it is working (j)

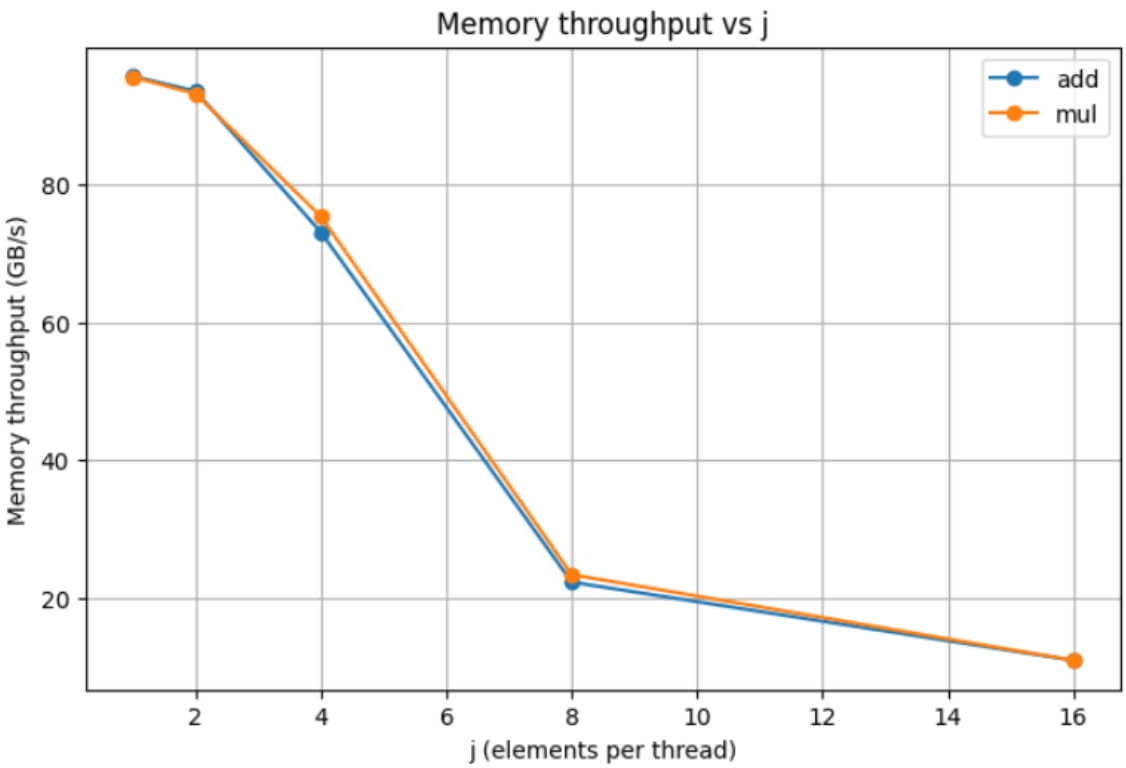


FIGURE 3.3: Execution speed (throughput) of the GPU depending of the number of element on which it is working (j)

We can see that as 'j' increases, the execution time rises sharply and performance (GB/s) drops dramatically. This can be explained by the fact that each thread processes more elements (than necessary), which proportionally reduces the total number of threads launched on the GPU. However, a GPU requires a very large number of available threads to operate optimally, to mask the existence of memory latency and to allow the GPU to function to its full potential.

## 3.3 Roofline analysis

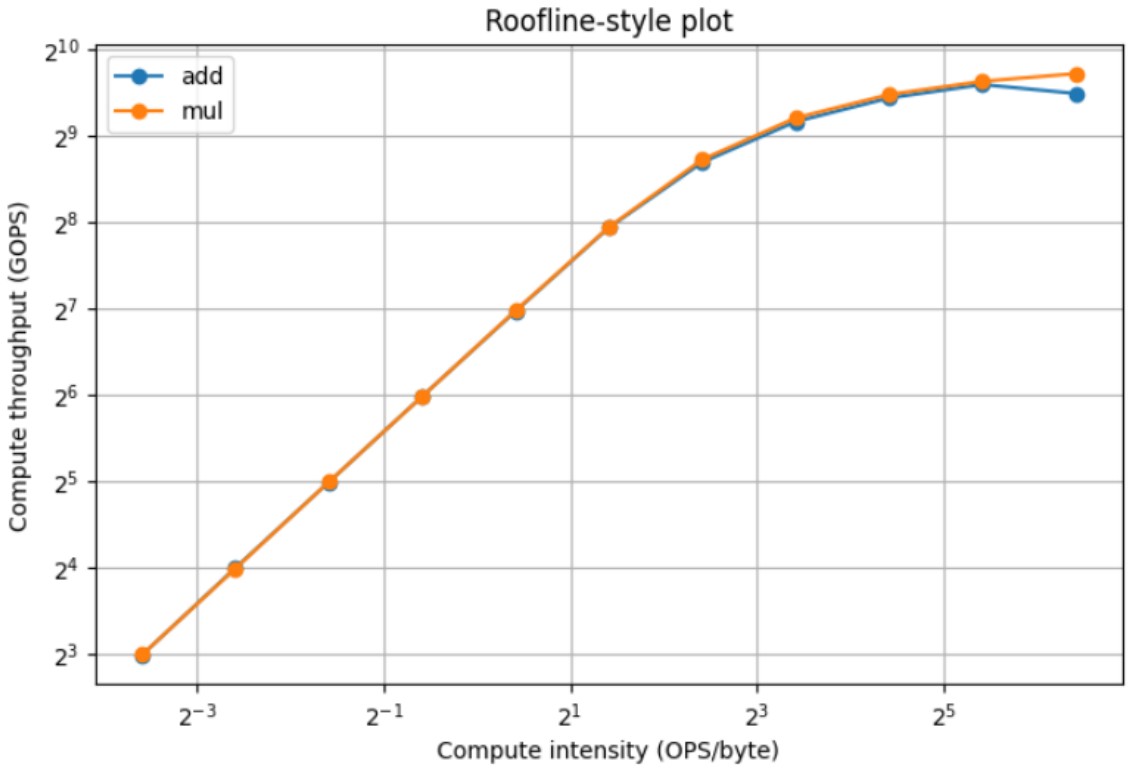


FIGURE 3.4: omputation performance as a function of the computational intensity

When constructing the roofline model, a consistent graph is observed for both operations. At low intensity, the relationship is linear, which indicates that the core is memory-bound. Performance increases with intensity before converging to a value at high intensity. This plateau marks a compute-bound region (it is no longer the number of operations that sets the limit for the core, but rather the compute units operating at their full potential)

# 4 Conclusion

As part of this project, we analysed two operations on floating-point data using different variations within a GPU kernel. The implications for both operations proved to be similar.

Finally, we were able to identify the limitations of this type of kernel, particularly in terms of memory, across various applications. Although a GPU is more powerful than a CPU, this application must be suitable for a large number of simultaneous operations and the computational intensity must also be optimal.